

Construyendo una aplicación Shiny para Finanzas

Del cálculo financiero a una interfaz interactiva

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)

Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-10-05

Objetivo de la clase

En esta clase construiremos paso a paso una aplicación **Shiny** que permita explorar un portafolio de cinco activos financieros. El usuario podrá modificar los pesos del portafolio y observar el retorno esperado, la volatilidad y el ratio de Sharpe.

La aplicación no reemplaza el análisis financiero: lo convierte en una interfaz para experimentar, formular preguntas y comunicar resultados.

Resultado final

Al terminar tendremos una aplicación con:

- controles para cambiar los pesos de SPY, EFA, IJS, EEM y AGG;
- normalización automática de los pesos para que sumen 100 %;
- métricas financieras calculadas en tiempo real;
- gráfico riesgo-retorno;
- comparación contra un portafolio equiponderado;
- botones para cargar una estrategia conservadora, balanceada o agresiva.

Idea central: en Shiny, la interfaz (ui) describe lo que el usuario ve y el servidor (server) contiene los cálculos que reaccionan a sus acciones.

Preparación

Instala los paquetes una sola vez:

```
install.packages(c("shiny", "readr", "dplyr", "tidyr", "ggplot2", "scales"))
```

La aplicación usará el archivo:

```
clases_ia/data/portafolio_retornos.csv
```

La versión ejecutable de la clase está en:

```
clases_ia/shiny_portafolio/app.R
```

Para abrirla, ejecuta `app.R` desde Positron o RStudio y presiona **Run App**.

Fase 1 — La aplicación mínima

Antes de trabajar con finanzas, construimos la aplicación más pequeña posible:

```
library(shiny)

ui <- fluidPage(
  titlePanel("Mi primera aplicación Shiny"),
  p("Esta aplicación todavía no calcula nada.")
)
```

```
server <- function(input, output, session) {  
}  
  
shinyApp(ui = ui, server = server)
```

Qué aprendemos

- `fluidPage()` crea una página adaptable.
- `titlePanel()` agrega un título.
- `server` será el lugar donde vivirán los cálculos.
- `shinyApp()` conecta la interfaz con el servidor.

En esta primera fase no hay inputs ni outputs. Solo comprobamos que Shiny está instalado y que podemos levantar una aplicación.

Fase 2 — Agregar una entrada y una salida

Ahora agregamos un deslizador y un texto que muestra su valor.

```
library(shiny)  
  
ui <- fluidPage(  
  titlePanel("Explorando un peso de portafolio"),  
  sliderInput(  
    inputId = "peso_spy",  
    label = "Peso de SPY (%)",  
    min = 0,  
    max = 100,  
    value = 40,  
    step = 1  
  ),  
  textOutput("mostrar_peso")  
)  
  
server <- function(input, output, session) {  
  output$mostrar_peso <- renderText({  
    paste("El peso seleccionado es", input$peso_spy, "%")  
  })  
}  
  
shinyApp(ui = ui, server = server)
```

La regla de reactividad

`input$peso_spy` contiene el valor actual del deslizador. Cuando el usuario lo mueve, Shiny vuelve a ejecutar el bloque `renderText()` y actualiza la salida.

La relación es:

```
input$peso_spy → renderText() → output$mostrar_peso
```

Fase 3 — Cargar los datos financieros

Los datos deben cargarse una sola vez, antes de construir la interfaz:

```
library(shiny)
library(readr)

datos <- read_csv("../data/portafolio_retornos.csv", show_col_types = FALSE)

head(datos)
```

La columna `fecha` identifica el período y las otras columnas contienen retornos mensuales.

```
activos <- c("SPY", "EFA", "IJS", "EEM", "AGG")
retornos <- as.matrix(datos[activos])

retornos_medios <- colMeans(retornos)
covarianzas <- cov(retornos)
```

En esta clase anualizamos los retornos mensuales usando:

```
retorno_anual <- retorno_mensual * 12
volatilidad_anual <- volatilidad_mensual * sqrt(12)
```

Fase 4 — Escribir las funciones financieras

Separar los cálculos de la interfaz facilita las pruebas y la explicación.

```
calcular_metricas <- function(pesos, retornos_medios, covarianzas,
                              tasa_libre = 0.02) {
  retorno <- sum(pesos * retornos_medios) * 12

  varianza <- as.numeric(t(pesos) %*% covarianzas %*% pesos)
  volatilidad <- sqrt(varianza) * sqrt(12)

  sharpe <- (retorno - tasa_libre) / volatilidad

  list(
    retorno = retorno,
    volatilidad = volatilidad,
    sharpe = sharpe
  )
}
```

La función implementa tres ideas:

1. El retorno del portafolio es el promedio ponderado de los retornos.
2. La volatilidad depende de la matriz de covarianzas.
3. El ratio de Sharpe compara el exceso de retorno con el riesgo.

También creamos una función para normalizar pesos:

```
normalizar_pesos <- function(pesos) {  
  if (sum(pesos) == 0) {  
    return(rep(1 / length(pesos), length(pesos)))  
  }  
  
  pesos / sum(pesos)  
}
```

Esto permite que el usuario mueva libremente los controles, pero el cálculo siempre trabaje con pesos que suman 1.

Fase 5 — Construir la interfaz financiera

Para evitar repetir código, definimos una función que crea un deslizador:

```
crear_slider <- function(activo, valor) {  
  sliderInput(  
    inputId = paste0("peso_", tolower(activo)),  
    label = paste0("Peso de ", activo, " (%)" ),  
    min = 0,  
    max = 100,  
    value = valor,  
    step = 1  
  )  
}
```

La interfaz completa puede organizarse en dos columnas:

```
ui <- fluidPage(  
  titlePanel("Laboratorio de Portafolios"),  
  
  sidebarLayout(  
    sidebarPanel(  
      h4("Pesos del portafolio"),  
      crear_slider("SPY", 40),  
      crear_slider("EFA", 20),  
      crear_slider("IJS", 10),  
      crear_slider("EEM", 10),  
      crear_slider("AGG", 20),  
      actionButton("conservador", "Conservador"),  
      actionButton("balanceado", "Balanceado"),  
    ),  
  )  
)
```

```

    actionButton("agresivo", "Agresivo")
  ),

  mainPanel(
    h3("Resultados"),
    textOutput("pesos_finales"),
    tableOutput("metricas"),
    plotOutput("grafico_riesgo_retorno", height = "500px")
  )
)
)
)

```

Fase 6 — Leer los controles de forma reactiva

En el servidor reunimos los valores de los cinco sliders:

```

pesos_reactivos <- reactive({
  pesos <- c(
    input$peso_spy,
    input$peso_efa,
    input$peso_ijs,
    input$peso_eem,
    input$peso_agg
  )

  normalizar_pesos(pesos)
})

```

Una expresión `reactive()` calcula un valor que será reutilizado por varias salidas. Si cambia cualquier slider, se actualiza `pesos_reactivos()` y Shiny recalcula solamente lo necesario.

Luego calculamos las métricas:

```

metricas_reactivas <- reactive({
  calcular_metricas(
    pesos = pesos_reactivos(),
    retornos_medios = retornos_medios,
    covarianzas = covarianzas
  )
})

```

Observa la diferencia entre una variable normal y una expresión reactiva:

```

pesos_reactivos()      # Se ejecuta llamando a la función
metricas_reactivas()   # También se consulta con paréntesis

```

Fase 7 — Mostrar resultados

```
output$pesos_finales <- renderText({
  pesos <- round(pesos_reactivos() * 100, 1)
  nombres <- paste(activos, pesos, "%", sep = " ")
  paste("Pesos normalizados:", paste(nombres, collapse = " | "))
})

output$metricas <- renderTable({
  metricas <- metricas_reactivas()

  data.frame(
    Métrica = c("Retorno anual", "Volatilidad anual", "Ratio de Sharpe"),
    Valor = c(
      paste0(round(metricas$retorno * 100, 2), "%"),
      paste0(round(metricas$volatilidad * 100, 2), "%"),
      round(metricas$sharpe, 3)
    ),
    check.names = FALSE
  )
})
```

La salida no modifica los cálculos. Solo transforma los resultados en un formato comprensible para la persona que usa la aplicación.

Fase 8 — Visualizar riesgo y retorno

Primero generamos una nube de portafolios aleatorios para tener un contexto.

```
set.seed(42)

simulacion <- replicate(3000, {
  pesos <- normalizar_pesos(runif(length(activos)))
  metricas <- calcular_metricas(pesos, retornos_medios, covarianzas)

  c(
    retorno = metricas$retorno,
    volatilidad = metricas$volatilidad,
    sharpe = metricas$sharpe
  )
})

simulacion <- as.data.frame(t(simulacion))
```

Después marcamos el portafolio elegido por el usuario:

```

output$grafico_riesgo_retorno <- renderPlot({
  elegido <- metricas_reactivas()

  ggplot(simulacion, aes(x = volatilidad, y = retorno, color = sharpe)) +
    geom_point(alpha = 0.35, size = 1.5) +
    geom_point(
      data = data.frame(
        volatilidad = elegido$volatilidad,
        retorno = elegido$retorno
      ),
      aes(x = volatilidad, y = retorno),
      color = "red",
      size = 5
    ) +
    scale_x_continuous(labels = scales::percent) +
    scale_y_continuous(labels = scales::percent) +
    labs(
      title = "Riesgo y retorno de portafolios",
      subtitle = "El punto rojo corresponde a tu selección",
      x = "Volatilidad anual",
      y = "Retorno anual",
      color = "Sharpe"
    ) +
    theme_minimal()
})

```

La nube representa alternativas posibles. El punto rojo representa la decisión del usuario. La aplicación ayuda a interpretar la posición relativa del portafolio, pero no convierte una estimación histórica en una garantía de retorno futuro.

Fase 9 — Botones y valores predefinidos

Los botones permiten comenzar desde estrategias reconocibles.

```

observeEvent(input$conservador, {
  updateSliderInput(session, "peso_spy", value = 15)
  updateSliderInput(session, "peso_efa", value = 10)
  updateSliderInput(session, "peso_ijs", value = 5)
  updateSliderInput(session, "peso_eem", value = 5)
  updateSliderInput(session, "peso_agg", value = 65)
})

observeEvent(input$balanceado, {
  updateSliderInput(session, "peso_spy", value = 30)
  updateSliderInput(session, "peso_efa", value = 15)
  updateSliderInput(session, "peso_ijs", value = 10)
})

```

```

updateSliderInput(session, "peso_eem", value = 10)
updateSliderInput(session, "peso_agg", value = 35)
})

observeEvent(input$agresivo, {
  updateSliderInput(session, "peso_spy", value = 40)
  updateSliderInput(session, "peso_efa", value = 25)
  updateSliderInput(session, "peso_ijs", value = 15)
  updateSliderInput(session, "peso_eem", value = 15)
  updateSliderInput(session, "peso_agg", value = 5)
})

```

`observeEvent()` escucha una acción. En este caso, la acción es hacer clic en un botón y la respuesta consiste en actualizar los cinco controles.

Fase 10 — Reflexión financiera

Responde las siguientes preguntas usando la aplicación:

1. ¿Qué ocurre con la volatilidad cuando aumenta el peso de AGG?
2. ¿Un portafolio con mayor retorno siempre tiene mayor ratio de Sharpe?
3. ¿Qué importancia tiene la correlación entre los activos?
4. ¿Qué diferencias observas entre una estrategia conservadora y una agresiva?
5. ¿Qué supuestos estamos haciendo al usar promedios y covarianzas históricas?

La aplicación es un instrumento para experimentar con estas preguntas. Las respuestas deben combinar el resultado numérico con una interpretación financiera.

Ejercicio

Modifica la aplicación para agregar:

- un campo para cambiar la tasa libre de riesgo;
- una tarjeta que indique cuál es el activo con mayor peso;
- una tabla con el retorno y la volatilidad de cada activo;
- un botón que restablezca todos los sliders a cero y evite errores;
- un segundo marcador para un portafolio de referencia.

Checklist técnico

- ☐ La aplicación abre desde `app.R`.
- ☐ Los cinco sliders modifican los resultados.
- ☐ Los pesos normalizados suman 100 %.
- ☐ El retorno, la volatilidad y Sharpe cambian al mover un slider.
- ☐ El gráfico muestra la simulación y el portafolio elegido.

- ☐ Los botones cargan estrategias distintas.
- ☐ El código está separado en datos, funciones, interfaz y servidor.

Idea clave

Una aplicación Shiny no es solamente un conjunto de controles. Es un modelo financiero conectado a una interfaz reactiva:

```
datos → funciones financieras → reactive() → outputs → interpretación
```

El valor de la aplicación está en hacer visible la relación entre una decisión de inversión y sus consecuencias en riesgo y retorno.